Digital Forensics & Steganography Complete CTF Field Guide

File identification • .bin / .exe / .txt analysis • Image steganography
Password bypass tricks • xxd deep-dive • Flag-hunting checklist

---

Compiled for: Sujal

SOC Analyst (L1) track · CTF / Jeopardy Prep · Digital Forensics Domain

Hinglish practical notes — har command ke saath real example aur "kab use karna hai" wali soch.

Is Guide Mein Kya Hai

1 File Identification — Sabse Pehla Step

2 Unknown / .bin File Deep Analysis

3 .exe / PE Binary Analysis

4 .txt File Analysis — Hidden Encoding

5 Image Steganography — Full Workflow

6 Password Bypass — Steghide / Zip / John

7 xxd Deep Dive — Hex Editing Mastery

8 Flag Format Cheat-Sheet

9 Universal Deep-Analysis Checklist

10 Advanced Tricks & Edge Cases

1 · File Identification — Sabse Pehla Step

Kabhi bhi extension pe bharosa mat karo — file ka asli type hamesha magic bytes se pata chalta hai.

Jab bhi koi unknown file mile (chahe .bin ho, .jpg ho, ya bina extension ke), sabse pehla kaam hai uska **real type** confirm karna. Challenge author aksar extension badal ke ya hata ke file deta hai — isliye **content** dekho, naam nahi.

Step-by-step Practical

```
# 1) File type turant check karo
file mystery.bin

# 2) Pehle 32 bytes hex me dekho (magic byte check)
xxd -l 32 mystery.bin

# 3) Readable text dhoondo (min length 6, printable only)
strings -n 6 -a mystery.bin | less

# 4) Metadata (agar image/pdf/office file ho)
exiftool mystery.bin
```

Magic Bytes Cheat-Table (xxd me pehle bytes)

Hex Bytes	Matlab / File Type
`89 50 4E 47`	PNG image (.png)
`FF D8 FF`	JPEG image (.jpg / .jpeg)
`47 49 46 38`	GIF image (.gif)
`25 50 44 46`	PDF document — ASCII "%PDF"
`50 4B 03 04`	ZIP / DOCX / XLSX / PPTX / JAR / APK (sab ZIP-based hain)
`52 61 72 21`	RAR archive
`37 7A BC AF`	7-Zip archive
`1F 8B`	GZIP (.gz)
`42 4D`	BMP image
`7F 45 4C 46`	ELF binary (Linux executable) — ASCII "\x7fELF"
`4D 5A`	PE / Windows .exe / .dll — ASCII "MZ"
`52 49 46 46 ... 57 41 56 45`	WAV audio
`49 44 33`	MP3 with ID3 tag
`00 00 00 18 66 74 79 70`	MP4 video

☞ *Trick: agar **file** command kuch ajeeb bole (jaise "data") aur extension kuch aur ho, toh samajh jao — header manually corrupt/patch kiya gaya hai. Yahi CTF ka favourite trick hai.*

2 · Unknown / .bin File — Deep Analysis

Jab file 'file' command bhi identify na kar paaye — tab yeh poora flow follow karo.

Poora Flow (order matters)

```
# 1) Basic recon
file suspicious.bin
ls -la suspicious.bin # size check karo

# 2) Embedded files dhoondo (bahut important — 90% bin challenges yaha solve hote)
binwalk suspicious.bin
binwalk -e suspicious.bin # -e = auto extract embedded files

# 3) File-carving (agar binwalk kuch na de)
foremost -i suspicious.bin -o carved_output/

# 4) Entropy check — high entropy = encrypted/compressed data
binwalk -E suspicious.bin

# 5) Hex dump poori file dekhne ke liye
xxd suspicious.bin | less
xxd suspicious.bin > dump.txt # file me save karke padho
```

Repeating Patterns / XOR Detection

Agar hex dump me koi 2-8 byte ka pattern baar-baar repeat ho raha ho, toh yeh usually **XOR-repeating-key encryption** ka sign hai (key length wahi hai jitna pattern lamba hai). Isko break karne ke liye:

- **Known-plaintext / crib-dragging:** agar tumhe pata hai flag format ctf7{ hai, toh us known text ko cipher ke saath XOR karke candidate key nikaalo.
- **CyberChef "XOR Brute Force"** recipe — sab 1-byte keys try karke output me readable text dhoondta hai.
- **Frequency analysis:** jo byte sabse zyada repeat ho rahi ho, wo usually space (0x20) ya null (0x00) plaintext ka XOR hoti hai.
- **Kasiski examination:** repeated n-gram ke beech ka distance nikaal ke unka GCD lo — wahi key-length ka strong hint hota hai.

```
# Python me quick XOR brute-force (single byte key)
data = open('file.bin','rb').read()
for k in range(256):
out = bytes(b ^ k for b in data)
if b'ctf7{' in out:
print(k, out)
```

☞ *Agar single-byte XOR se kuch na mile, key-length 2/3/4/5 try karo — CyberChef 'Multi XOR Brute Force' ya apna Python loop likh ke.*

3 · .exe / PE Binary Analysis

Reverse engineering + static analysis — bina execute kiye jaan■■■■ nikaalna.

Golden Rule: Unknown .exe ko kabhi apne real machine par run mat karo. Static analysis (bina run kiye) se hi zyadatar CTF flags mil jaate hain. Agar dynamic zaroori ho toh sirf isolated VM / sandbox me karo.

Static Analysis Commands

```
file program.exe
strings -n 6 -a program.exe | grep -iE 'flag|ctf|http|password'
strings -el program.exe # -el = UTF-16LE strings (Windows me common)
exiftool program.exe # compile time, metadata
# PE header details (Linux se)
objdump -x program.exe | less
readelf -h program.exe # agar ELF (Linux binary) hai
# Detect packing / obfuscation
die program.exe # Detect It Easy (packer/compiler detect)
```

GUI / Deeper Tools

Command / Tool	Kab & kyun use kare
`Detect It Easy (DIE)`	Packer, compiler, language detect karta hai — sabse pehle chalao.
`Ghidra`	Free NSA decompiler — pseudo-C code me dekh sakte ho binary ka logic.
`IDA Free`	Disassembler — assembly level analysis ke liye industry standard.
`x64dbg / OllyDbg`	Dynamic debugger — sirf sandbox/VM me use karo.
`PEview / CFF Explorer`	PE header, sections, imports/exports GUI me dekhna.
`UPX -d program.exe`	Agar UPX-packed ho toh yeh command unpack kar deta hai.

☞ *Flag dhoondte waqt: pehle **strings + grep** try karo (sabse fast). Agar flag encrypted/encoded dikhe andar Ghidra me, toh us function ka decompiled logic padho — usually simple XOR ya string-compare check hota hai.*

4 · .txt File Analysis — Hidden Encoding

Text file 'simple' lagti hai, lekin encoding ke andar encoding chupi ho sakti hai.

Checklist

- **Hidden/invisible characters:** cat -A file.txt — tabs (^I), trailing spaces, CRLF (^M) dikhayega.
- **Base64 lagta hai?** (letters+numbers+ / + = padding) → base64 -d file.txt
- **Hex lagta hai?** (sirf 0-9 a-f, even length) → echo <text> | xxd -r -p
- **Binary (0/1) lagta hai?** → 8-bit groups leke ASCII me convert karo.
- **ROT13 / Caesar cipher** → tr 'A-Za-z' 'N-ZA-Mn-za-m' < file.txt
- **Morse code, Braille, NATO alphabet** — CyberChef me ready-made recipes hain.
- **Zero-width character steganography:** text me invisible Unicode chars (U+200B, U+200C) chupe ho sakte hain jo binary data encode karte hain — normal eye se nahi dikhte!

Practical Commands

```
cat -A file.txt | less # hidden chars dikhao
hexdump -C file.txt | less # raw bytes (invisible unicode bhi dikhega)
# Zero-width steg check
grep -P '[\x{200B}\x{200C}\x{200D}\x{FEFF}]' file.txt
# Base64 decode
base64 -d file.txt > decoded_output
# Quick 'try everything' approach
cat file.txt | base64 -d 2>/dev/null | strings
```

☞ *Sabse fast tareeka: file ka content **CyberChef** me daal ke "Magic" wand (baking-tray icon) use karo — yeh khud detect karke recipe suggest karta hai (base64, hex, gzip, sab try karta hai).*

5 · Image Steganography — Full Workflow

Sabse zyada CTF forensics challenges images me hi hote hain — is order me chalo.

Step 1 — Basic Recon

```
file secret.jpg # real type confirm (extension match karta hai?)
exiftool secret.jpg # comments, GPS, software, hidden metadata fields
strings -n 6 -a secret.jpg | grep -iE 'flag|ctf'
```

Step 2 — Appended / Embedded Data (bahut common trick)

Image files ka ek fixed "end marker" hota hai (JPEG: FF D9, PNG: IEND chunk). Uske BAAD bhi extra data append kiya ja sakta hai (jaise ek chhupa hua ZIP ya text) — image viewer usko ignore kar deta hai, lekin file me wo hota hai.

```
binwalk secret.jpg # dikhayega agar EOF ke baad kuch aur file hai
binwalk -e secret.jpg # extract kar dega automatically
# Manually bhi kar sakte ho:
xxd secret.jpg | grep -n 'ffd9' # JPEG end-marker dhoondo
dd if=secret.jpg bs=1 skip=<offset> of=hidden_data # marker ke baad se sab nikaalo
```

Step 3 — LSB Steganography (PNG / BMP)

Least Significant Bit steg — pixel color values ke last bit me data chupaya jaata hai. Insaan ki aankh se dikhta nahi, lekin tool se extract hota hai.

```
zsteg secret.png # PNG/BMP ke liye best LSB-detection tool
zsteg -a secret.png # sab possible combinations try karta hai
# GUI visual tool — bit-planes alag-alag dikhata hai
java -jar stegsolve.jar
```

Step 4 — steghide (JPG / BMP / WAV)

```
steghide info secret.jpg # kya kuch embedded hai check karo
steghide extract -sf secret.jpg # passphrase maangega
steghide extract -sf secret.jpg -p "" # empty password bhi try karo pehle
```

Step 5 — Corrupted / Patched Image Puzzles

- PNG file corrupted na khule? → pngcheck -v file.png se error dekho.
- Classic trick: PNG ke IHDR chunk me **height** byte manually chota kar diya jaata hai — isse image crop dikhti hai. Hex editor me IHDR ke baad wali height value badhao (CRC bhi recalc karna padega — CyberChef "PNG Decompress"/online tool use karo, ya pngcheck se sahi values lo).
- Width/Height dono galat ho sakte hain — real dimensions file-size se estimate karke try karo.
- identify -verbose file.png (ImageMagick) — actual vs claimed properties compare karo.

6 · Password Bypass — Steghide / ZIP / John

Jab extraction 'passphrase' maange aur pata na ho — yeh cracking flow follow karo.

Tumne bola ki extraction 'pass mangta hai' — yeh 2 cases me hota hai: **(a)** steghide-type image jisme passphrase chahiye, ya **(b)** embedded ZIP/RAR jo password-protected hai. Dono ke liye alag approach:

Case A — steghide Passphrase Bruteforce

```
# stegseek — stegcracker se bahut fast (recommended, use isi ko)
stegseek secret.jpg /usr/share/wordlists/rockyou.txt

# purana tool (dh■m■ par kaam karta hai)
stegcracker secret.jpg /usr/share/wordlists/rockyou.txt

# Pehle hamesha empty-password aur common guesses try karo
steghide extract -sf secret.jpg -p ""
steghide extract -sf secret.jpg -p "password"
```

Case B — Password-Protected ZIP / RAR

```
# ZIP ke liye — fcrackzip (fast, wordlist ya bruteforce dono)
fcrackzip -D -u -p /usr/share/wordlists/rockyou.txt secret.zip

# John the Ripper approach (zyada reliable, GPU-friendly)
zip2john secret.zip > zip.hash
john --wordlist=/usr/share/wordlists/rockyou.txt zip.hash
john --show zip.hash

# RAR ke liye
rar2john secret.rar > rar.hash
john --wordlist=rockyou.txt rar.hash
```

Case C — PDF ya Office file password

```
pdf2john secret.pdf > pdf.hash
john --wordlist=rockyou.txt pdf.hash

# ya qpdf se seedha remove (agar sirf owner-password hai, open-password nahi)
qpdf --password=guessed_pass --decrypt secret.pdf output.pdf
```

☞ *Wordlist na mile toh: locate rockyou.txt ya apt install wordlists (Kali me pehle se hota hai /usr/share/wordlists/rockyou.txt.gz — pehle gunzip karna padega). Challenge ke **hints/description** me hi kabhi kabhi password diya hota hai — usko miss mat karo!*

7 · xxd Deep Dive — Hex Editing Mastery

xxd sirf 'dekhne' ka tool nahi — isse edit/patch/repair bhi hota hai.

Output Padhna Seekho

```
00000000: 8950 4e47 0d0a 1a0a 0000 000d 4948 4452 .PNG........IHDR
# ^col1(offset) ^col2 = 16 bytes hex (2-2 group me) ^col3 = ASCII view
```

Column 1 = byte offset (hex me), Column 2 = actual hex bytes, Column 3 = wahi bytes ASCII printable form me (non-printable = dot '.').

Zaroori Commands

```
xxd file.bin # poora hex dump
xxd -l 64 file.bin # sirf pehle 64 bytes
xxd -s 0x100 -l 32 file.bin # offset 0x100 se 32 bytes
xxd -s -32 file.bin # last 32 bytes (negative offset)

# HEX → BINARY convert karke wapas file banao (reverse mode)
xxd -r hexdump.txt restored.bin

# Sirf hex string se binary banao (koi offset column nahi)
echo '89504e47' | xxd -r -p > header.bin
```

Patching / Editing Bytes (Manual Repair)

Kaafi CTF challenges me file ka header jaan-bujh kar corrupt kiya hota hai (jaise PNG signature ka ek byte change kar diya). Fix karne ka tareeka:

```
# 1) Dekh lo galat byte kahan hai
xxd file.png | head -2

# 2) sed se specific hex byte replace karo, phir wapas binary banao
xxd file.png | sed '1s/1x50/1x89/' | xxd -r - fixed.png

# 3) ya Python se seedha (zyada reliable, recommended)
python3 -c "
data = bytearray(open('file.png','rb').read())
data[0] = 0x89 # correct PNG magic byte laga do
open('fixed.png','wb').write(data)"
```

☞ *Hamesha original file ki **copy** pe kaam karo — galti se overwrite ho gaya toh original evidence chala jaayega.*

8 · Flag Format Cheat-Sheet

Format pehle se pata ho toh grep/regex bahut fast ho jaata hai.

Format	Kis platform / context me
`ctf7{...}`	Is challenge/platform ka apna custom format — jo tumhe diya gaya hai (ctf7{...})
`flag{...}`	Sabse common generic CTF format
`CTF{...}`	Uppercase variant — platform-specific
`picoCTF{...}`	picoCTF platform
`HTB{...}`	HackTheBox challenges
`THM{...}`	TryHackMe challenges
`FLAG-xxxx-xxxx`	Kuch corporate/custom CTFs dash-separated bhi rakhte hain

Flag Dhoondne Ke Liye Universal grep

```
# Extracted / decoded content me directly flag search karo
grep -aoE 'ctf7\{[^}]*\}' output.txt
strings -a file.bin | grep -aoE '[a-zA-Z0-9_]+\{[^}]*\}'
# Binwalk-extracted folder me recursively search
grep -raoE 'ctf7\{[^}]*\}' _extracted/
```

☞ *Agar flag ka format pehle se pata hai (jaise **ctf7{}**), toh use **known-plaintext crib** ke roop me use karo — encryption/XOR tod-ne ke liye yeh sabse strong clue hota hai.*

9 · Universal Deep-Analysis Checklist

Kisi bhi mystery file ke saath is order me chalo — kabhi flag miss nahi hoga.

1. **file <filename>** — real type pehchano
2. **xxd -l 32 <filename>** — magic bytes verify karo, extension se match kar raha hai kya?
3. **strings -n 6 -a <filename> | grep -iE 'flag|ctf'** — sabse fast win
4. **exiftool <filename>** — metadata / comments / GPS / author fields check karo
5. **binwalk <filename>** phir **binwalk -e** — embedded files nikaalo
6. **binwalk -E** — entropy graph dekho (flat high = encrypted/compressed section hai)
7. Agar image hai → zsteg / steghide / stegsolve (Section 5 dekho)
8. Agar text/encoding lagta hai → base64 / hex / ROT13 / CyberChef Magic (Section 4)
9. EOF ke baad extra data check karo (appended ZIP/text — Section 5, Step 2)
10. Repeating byte pattern dikhe → XOR key-length nikaalo, crib-drag karo (Section 2)
11. Password maange → rockyou.txt + stegseek/john/fcrackzip (Section 6)
12. Kuch na mile → challenge description/hints dubara padho, filename khud bhi clue ho sakta hai

10 · Advanced Tricks & Edge Cases

Jab upar wala sab try ho chuka ho aur flag phir bhi na mile.

Polyglot Files

Ek file jo **ek saath do formats** ke rules follow karti hai (jaise ek file jo valid JPEG bhi hai aur valid ZIP bhi). file ek type bolega, lekin extension-change karke doosra type bhi khul sakta hai.

```
cp mystery.jpg mystery.zip
unzip mystery.zip # dekho kuch extract hota hai kya
```

Alternate Data Streams (NTFS / Windows)

```
dir /r file.txt # Windows CMD — hidden streams list karega
more < file.txt:hidden.txt # stream ka content padho
```

Git Repository Forensics

Agar ek .git folder mila hai (ya usko dhoondna hai), toh deleted commits aur history me flag chupa ho sakta hai:

```
git log --all --oneline # sab commits (deleted branches bhi)
git show <commit-hash>
git log -p # sab file-diffs history ke saath
git fsck --unreachable # dangling/deleted objects dhoondo
```

Memory Dumps

```
volatility -f memdump.raw imageinfo # profile detect karo pehle
volatility -f memdump.raw --profile=<P> pslist
volatility -f memdump.raw --profile=<P> cmdscan
```

Network Captures (PCAP)

```
wireshark capture.pcap # GUI analysis
tshark -r capture.pcap -Y 'http' # sirf HTTP traffic filter
tshark -r capture.pcap --export-objects http,extracted/ # files nikaalo
```

Audio Steganography

- **Spectrogram check** — Sonic Visualiser / Audacity me spectrogram view kholo, kabhi kabhi text/image waveform ke andar visually chupi hoti hai.
- **steghide** WAV files ke liye bhi kaam karta hai (Section 6 dekho).
- **SSTV / Morse tone** — agar beep pattern ho, decode karne ke liye online SSTV/Morse decoder use karo.

Compiled & maintained by Sujal — Digital Forensics domain, CTF prep reference.